

Architecture & Design Modules

Six building blocks that turn a business process into a governed, multi-agent system on Azure.

A1

Agent Architecture Patterns
Planner–Executor · ReAct · Supervisor · Hierarchical. Choose the control topology that fits the task.

A2

Multi-Agent Orchestration
LangGraph state graphs + Microsoft Agent Framework. Deterministic workflows meet autonomous agents.

A3

MCP & A2A Contracts
Model Context Protocol for tools/data. Agent-to-Agent for peer discovery & delegation.

A4

Retrieval Augmented Agents
Vector + hybrid retrieval, grounding, and citation loops to keep agents faithful to enterprise knowledge.

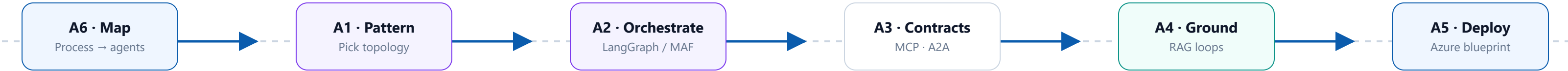
A5

Azure Infrastructure Blueprint
AKS, Azure OpenAI, AI Search, Key Vault, App Insights — the reference wiring for an agentic platform.

A6

Functional Mapping Workshop
Translate a business process into agents, tools, data contracts, and Human-in-the-Loop checkpoints.

HOW THE MODULES CONNECT



Four Control Topologies for Agents

Planner–Executor · ReAct · Supervisor · Hierarchical — pick the pattern that matches task shape, latency, and cost.

PATTERN 1

Planner–Executor

Deliberate plan first, then execute step by step.

```
graph LR; Goal[Goal] --> Planner[Planner  
Decomposes]; Planner --> Step1[Step 1]; Planner --> Step2[Step 2]; Planner --> StepN[Step N]; Step1 --> Executor[Executor  
Runs each step]; Step2 --> Executor; StepN --> Executor; Executor --> Result[Result]
```

Use when: tasks are decomposable, cost matters, and steps can be verified independently. **Trade-off:** plan quality caps outcomes; poor at surprises.

PATTERN 2

ReAct (Reason + Act)

Interleave thought, action, and observation in a loop.

```
graph LR; Thought[Thought] --> Action[Action  
Tool call]; Action --> Observation[Observation]; Observation -. "loop until confident" .-> Thought; Thought --> Answer[Answer]
```

Use when: the environment is uncertain and tool use is central. **Trade-off:** variable latency and tokens; needs strong stop conditions.

PATTERN 3

Supervisor (Router)

One coordinator dispatches to specialist workers.

```
graph TD; Supervisor[Supervisor  
Routes & joins] --> Research[Research]; Supervisor --> Write[Write]; Supervisor --> Code[Code]; Supervisor --> Review[Review]; Supervisor --> Finish[Finish]
```

PATTERN 4

Hierarchical (Teams of Teams)

Nested supervisors — each team owns a sub-domain.

```
graph TD; CEO[CEO Agent] --> Sales[Sales Lead]; CEO --> Ops[Ops Lead]; CEO --> Finance[Finance Lead]; Sales --> LeadGen[Lead-gen]; Sales --> Outreach[Outreach]; Ops --> Scheduler[Scheduler]; Ops --> Tickets[Tickets]; Finance --> Invoicing[Invoicing]; Finance --> Reporting[Reporting]
```

LangGraph & Microsoft Agent Framework

Two complementary runtimes: a state-graph engine for deterministic flows and an enterprise agent host for Azure-first deployments.

LANGGRAPH

Graph-based state machines for agents

Nodes = agents/tools · Edges = transitions · State = shared blackboard.

START

Router

Conditional edge

Researcher

Coder

Analyst

END

re-route on quality check

Shared State (TypedDict / Pydantic)

messages · task_plan · retrieved_docs · tool_results · next_agent

Every node reads/writes fields; reducers merge concurrent updates.

Why LangGraph

- Explicit control flow — easy to reason, test, and replay
- Checkpointing & time-travel debugging (Postgres/SQLite)
- Streaming events for UI; HITL as a first-class interrupt
- Runs anywhere Python runs — AKS, Container Apps, Functions

MICROSOFT AGENT FRAMEWORK

Enterprise agent host on Azure

Unifies Semantic Kernel + AutoGen · .NET & Python · first-class Azure OpenAI, AI Search, Foundry.

Channels · Copilot Studio · Web · Teams · Custom UI

Orchestrator · Workflows & Groups

Agent A

Skills · Tools

Agent B

Skills · Tools

Agent C

Skills · Tools

Agent D

Skills · Tools

Memory · Threads · Vector Store · Session State

Connectors · MCP Servers · OpenAPI Tools · Azure AI Search

Azure OpenAI · Foundry Models · Fine-tunes

Governance · Identity (Entra) · Tracing · Content Safety

When to choose MAF

- Regulated .NET / Azure-heavy enterprise estates
- Native Entra ID, Purview, App Insights integration
- Copilot Studio / Teams distribution out of the box
- Interop with LangGraph via MCP tools & A2A calls

Track A — Architecture & Design

A2 · Orchestration

MCP & A2A — the Two Contracts of an Agent Stack

MCP wires agents to tools and data. A2A wires agents to each other. Together they make agent systems composable across vendors.

MCP

Model Context Protocol · vertical contract

Agent ↔ Tools & Data

Standardised JSON-RPC surface for exposing tools, resources, and prompts to any LLM host.

```
graph LR; Client["MCP Client<br/>Agent / Host"] <-->|JSON-RPC 2.0| Servers; subgraph Servers; S1["MCP Server: CRM<br/>create_lead · get_account"]; S2["MCP Server: SAP<br/>post_invoice · list_orders"]; S3["MCP Server: Files<br/>SharePoint · Blob"]; S4["MCP Server: DB<br/>query · schema"]; end; Client --> S1; Client --> S2; Client --> S3; Client --> S4;
```

Core primitives

Primitive	Purpose	Example
Tools	Callable functions with typed args	create_ticket(subject, priority)
Resources	Read-only content the model can attach	sap://orders/2026-Q3
Prompts	Reusable templates shipped by server	invoice_dispute_template
Sampling	Server asks host to run an LLM call	Delegated summarisation

A2A

Agent-to-Agent Protocol · horizontal contract

Agent ↔ Agent

Peer discovery, capability negotiation, task delegation across trust boundaries.

```
graph LR; A["Agent A<br/>Publishes Agent Card"] <-->|A2A · HTTPS + SSE| B["Agent B<br/>Publishes Agent Card"]; linkStyle 0 stroke:#800080,stroke-width:2px; linkStyle 0 label:tasks · messages · artifacts;
```

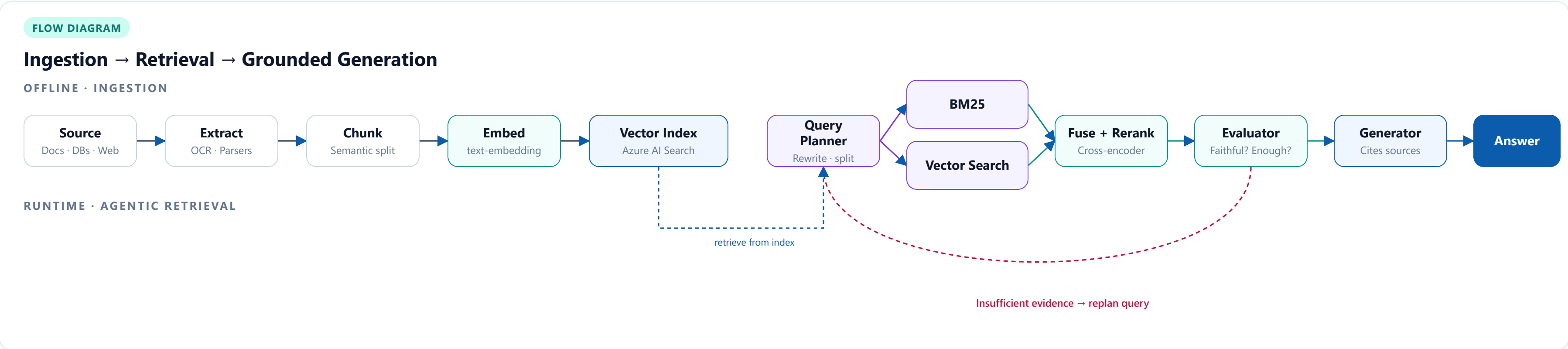
Delegation sequence

- Discover** — Agent A fetches /.well-known/agent.json from B
- Authenticate** — OAuth2 / OIDC · Entra token exchange
- Create task** — POST /tasks with skill + input parts
- Stream updates** — SSE: working → input-required → completed
- Collect artifacts** — files, structured JSON, cited sources
- Audit** — task id + trace persisted in App Insights

Rule of thumb: if it's a function inside your trust boundary → MCP. If it's another team's agent behind an API → A2A.

Retrieval Augmented Agents (Agentic RAG)

Beyond one-shot RAG: agents plan queries, evaluate evidence, and loop until the answer is grounded.



TECHNIQUES

Retrieval quality levers

Hybrid search — BM25 keyword + dense vectors, fused via RRF.

HyDE / expansions — synthesise hypothetical answers to broaden recall.

Metadata filters — tenant, ACL, date, document type at query time.

Contextual chunking — prepend section headings to each chunk.

Query decomposition — split multi-hop questions into sub-queries.

Reranking — cross-encoder or LLM-as-judge on top-K candidates.

Parent-doc / small-to-big — embed small chunks, feed larger context.

Citation enforcement — refuse answers without a supporting span.

GUARDRAILS

Faithfulness & security controls

Access controlRow-level ACLs propagated from source to index (Entra groups).

PII & content safetyPresidio + Azure Content Safety on ingest and on response.

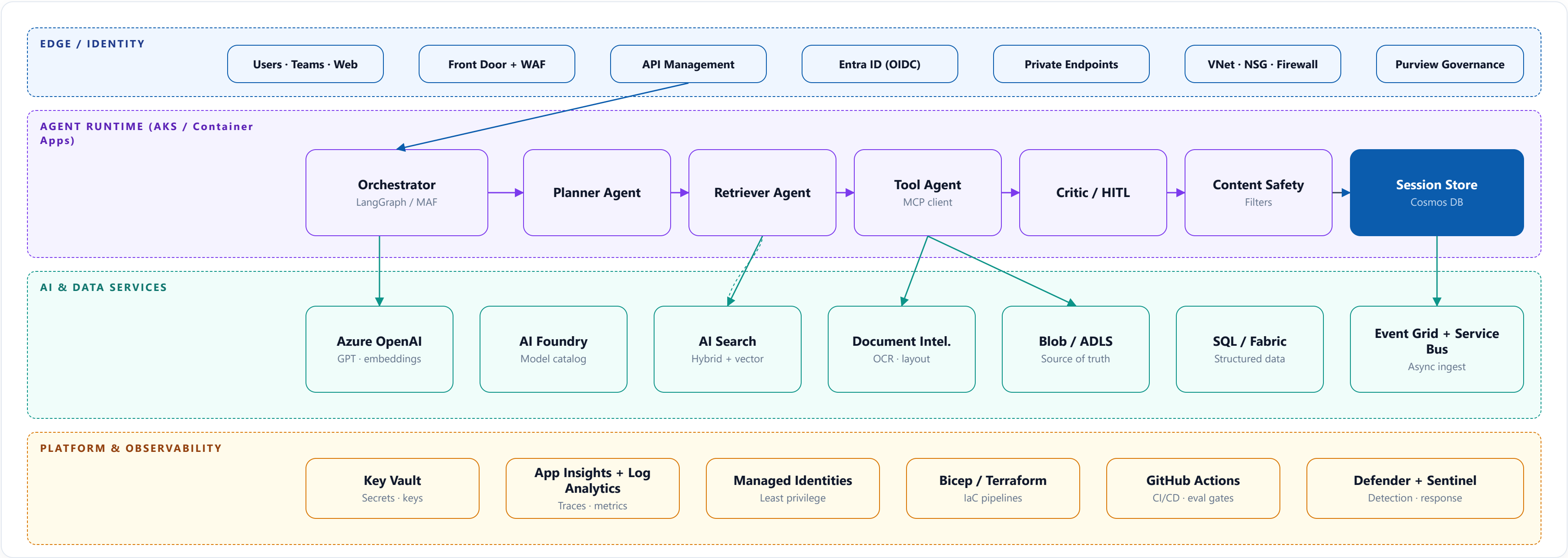
Groundedness scoreVerify every claim maps to a retrieved span; refuse if below threshold.

Prompt-injection defenseStrip instructions from retrieved text; sandbox tool arguments.

Freshness & TTLIncremental re-index; expire stale chunks; changed-doc invalidation.

Azure Infrastructure Blueprint for Agentic Solutions

A landing-zone-friendly wiring diagram — from user request to grounded model response, with governance baked in.



Deployment pattern: single subscription per environment · hub-spoke VNet · private-endpoint to every PaaS · tenant-scoped keys in Key Vault referenced by Managed Identity · Bicep modules per zone · GitHub Actions with eval-quality gate before promotion.

From Business Process to Agents & HITL

A repeatable workshop pattern: swimlane the process, decompose into agent-sized tasks, place Human-in-the-Loop where it matters.

WORKSHOP FLOW

Five steps · one afternoon per process

1 · Capture
As-is process + KPIs

2 · Decompose
Tasks · decisions · data

3 · Classify
Auto · Assist · Human

4 · Map to agents
Roles + tools + data

5 · Place HITL
Approvals · overrides

WORKED EXAMPLE

Purchase-order approval — mapped to agents + HITL

Requester
(Human)

Submit PO

Intake Agent

Parse + extract

Enrich vendor

Policy Agent

Check budget

Risk score

Approver
(HITL)

> \$50k?

Approve / reject

Trace: Requester submits PO → Intake Agent parses + enriches → Policy Agent checks budget & risk → if amount < threshold, auto-approve; otherwise Approver receives a summary card with cited evidence and 1-click approve/reject.

HITL RUBRIC

Where humans stay in the loop

Approve — before an action
High-value writes, external comms, irreversible ops.

Review — after a suggestion
Draft replies, code suggestions, generated content.

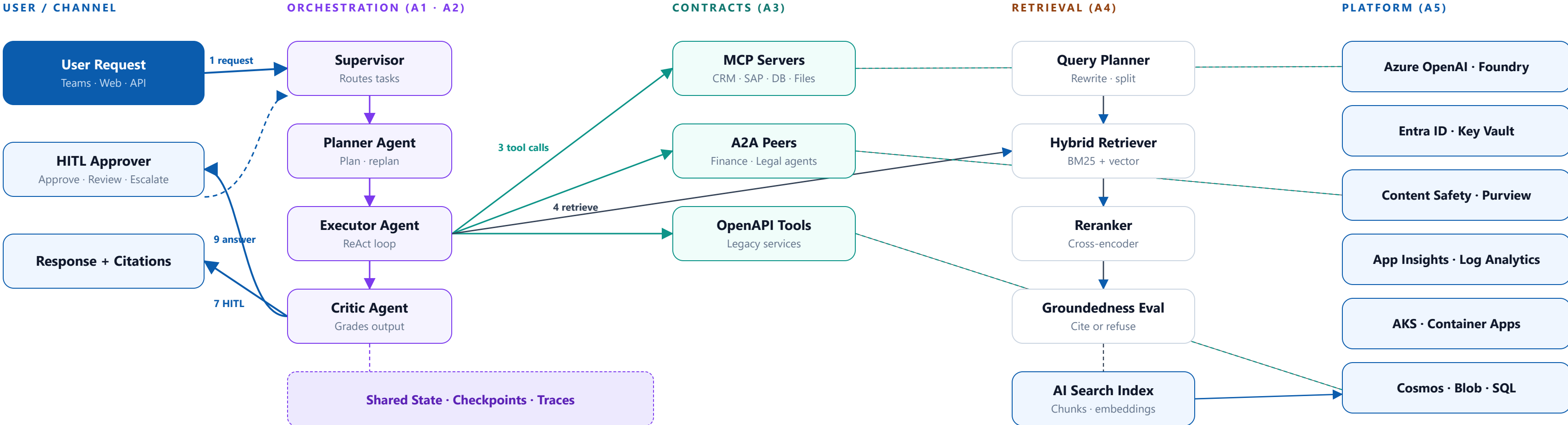
Escalate — on uncertainty
Low-confidence answers, novel cases, policy edge conditions.

Audit — always
Every decision, tool call, and citation is logged with actor + trace-id.

Guideline: automate the routine, augment the judgement calls, escalate the exceptions.

Putting It Together — One Request, End-to-End

A single wiring diagram showing how A1–A6 combine when a user issues a real request.



Read the diagram left → right:

1 User request enters via a channel · 2 Supervisor plans / executes / critiques via A1 pattern on A2 runtime · 3 Executor calls tools through **MCP** and peer agents through **A2A** · 4 Retriever grounds every claim on AI Search · 5–6 All traffic runs on the A5 Azure blueprint under Entra + Content Safety · 7 Critic escalates to **HITL** when policy or confidence demands · 8 Approver's decision re-enters shared state · 9 Final answer with citations returns to the user.